

Optimizing ML And DNN By Genetic Algorithms

^{1st} Zihao Dongfang

Department of Computing & Decision Sciences
Lingnan University
Hongkong SAR, China
zihadongfang@ln.hk

^{2nd} Tenglong Zeng

Department of Computing & Decision Sciences
Lingnan University
Hongkong SAR, China
tenglongzeng@ln.hk

Abstract—This study investigates the application of genetic algorithms (GA) for hyperparameter optimization in machine learning models, including Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, and Multilayer Perceptron (MLP). The performance of models with default settings, optimized using grid search, and optimized using GA is compared using the F1 macro score on the "Heart Disease" dataset from the UCI Machine Learning Repository. The results demonstrate that hyperparameter optimization, both through grid search and GA, generally improves model performance compared to default configurations. GA optimization consistently outperforms or matches grid search performance for all models tested. For GA-optimized MLP, a specialized initialization mechanism, crossover, and mutation operators are introduced to optimize both network structure and hyperparameters. The best GA-optimized MLP achieves the highest F1 macro score of 87% among all experiments, surpassing both default and grid search optimized models. The study highlights the potential of GA for automating hyperparameter tuning, efficiently exploring the search space, and finding near-optimal configurations. As the complexity of machine learning models grows, evolutionary algorithms like GA are likely to become increasingly important for hyperparameter optimization in the field of machine learning.

Index Terms—Genetic Algorithms, Machine Learning, Deep Neural Networks, Multi-layer Preceptron, Heart Disease

I. INTRODUCTION

In recent decades, machine learning and deep neural networks have become the core of the field of artificial intelligence, especially in tasks such as image recognition and natural language processing. However, choosing the best model configuration remains a challenge and requires significant knowledge and resources. In this context, genetic algorithm [1], as a heuristic optimization method, provides a powerful tool for automating and optimizing these decision-making processes. Genetic algorithms are algorithms inspired by natural biological evolution mechanisms, and they solve optimization problems by simulating biological inheritance, variation, natural selection and gene recombination processes. Starting from a randomly generated set of candidate solutions, the quality of the solution is gradually improved through an iterative process. In each generation, the algorithm will evaluate the performance of each candidate solution according to the fitness function, select the best solutions for reproduction, and generate new solutions through crossover and mutation operations, so as to explore the solution space and optimize the objective function. This algorithm shows unique advantages in

global optimization problems, in the field of machine learning and deep learning, GA is used for several optimization tasks, such as:

Hyperparameter optimization, the performance of ML and DNN models is highly dependent on the choice of hyperparameters. In hyperparameter optimization, an "individual" typically represents a specific set of hyperparameter configurations, while the entire "population" represents a collection of different configurations. The inter-algorithm generates new hyperparameter configurations by crossing and mutating, and iterates until it finds a solution that performs best or satisfies certain conditions. In SVM, genetic algorithms can be used to optimize kernel parameters and regularization parameters C. In neural networks, it can be used to determine the optimal network structure, learning rate, activation function, etc. In the decision tree can help select the depth of the decision tree, split criteria and other hyperparameters.

Feature selection: Selecting the most influential feature subset in high-dimensional data is crucial to improve model performance and reduce overfitting. [2] GA improves model generalization ability by optimizing feature subset selection. The development of effective coding strategy and the design of reasonable fitness function are conducive to the optimization of feature selection. For example, a feature subset is randomly generated in a random forest as the initial population, the model performance is evaluated, subsequent operations are carried out according to the subset performance, and then crossover, mutation, and iteration are performed. In this way, GA helps random forest models maintain high accuracy while reducing the number of features, simplifying the model and potentially reducing training and prediction time.

Neural network structure search (NAS) is mainly used to automatically explore and optimize the network structure, including the number of layers, the number of neurons in each layer, the selection of activation function and other parameters. [3] By simulating the process of natural evolution, GA iteratively generates, evaluates, selects, crosses and mutates the network architecture, aiming to find the optimal or near optimal architecture to improve the model performance.

Genetic algorithms perform well in optimizing high-dimensional, non-convex problems in machine learning and deep learning, despite high computational costs. However, with the advancement of computing technology and parallel computing, its potential in automated model design and opti-

mization is gradually being tapped.

This study focuses on utilizing genetic algorithms for hyperparameter optimization in machine learning algorithms and deep learning models (Multi layer perceptron) to enhance their performance and efficiency in predicting heart disease. Specially, we introduce special crossover and mutation operator for search architecture of MLP. Further more, we introduce diversity and elitism to balance exploration and exploitation.

The research is conducted on the "**Heart Disease**" dataset, with a primary emphasis on a subset of 14 attributes from the dataset's 76 attributes, particularly the "Cleveland" database. [4] Initially, common algorithms like Support Vector Machines, Random Forests, and simple deep learning networks are trained and evaluated on the dataset to establish baseline performance. Subsequently, genetic algorithms are introduced to refine the models further, aiming to overcome local optima and achieve improved prediction accuracy and efficiency. We will also compare this method with the conventional **Grid Search** method to search for the optimal hyperparameters. Through genetic algorithm hyperparameter optimization, the study developed superior models with enhanced performance and accuracy in heart disease prediction.

II. RELATED WORK

A. Genetic Algorithms

The fundamental idea of genetic algorithms begins with a set of initial individuals representing possible solutions to a problem, known as a "population." Through an iterative process, or "evolution," it searches for the optimal solution. Each individual is typically represented by a string (which can be binary or other encoding methods), known as a "chromosome." A chromosome represents a potential solution to the problem.

Here is the basic process of a genetic algorithm (GA) and an introduction to the mainstream selection, crossover, and mutation methods:

Initialize the population: Randomly generate a fixed number of individuals to form the initial population. Each individual is represented by a chromosome, which is typically encoded as a binary string or using other encoding methods.

Evaluate fitness: Calculate the fitness value of each individual based on the problem's objective function. The fitness value reflects the quality of the individual as a solution to the problem.

Selection: Select a portion of the individuals as parents to produce the next generation based on their fitness values. Common selection methods include:

- **Roulette Wheel Selection:** The probability of an individual being selected is proportional to its fitness value [5].
- **Tournament Selection:** Randomly select a fixed number of individuals and choose the one with the highest fitness value as a parent [6].
- **Rank Selection:** Rank the individuals based on their fitness values and determine the selection probability according to their rank [7].

Crossover: Pair the selected parent individuals and produce new offspring individuals through crossover operations. Common crossover methods include:

- **Single-Point Crossover:** Randomly select a position on the chromosome and exchange the gene segments after that position [8].
- **Two-Point Crossover:** Randomly select two positions on the chromosome and exchange the gene segments between these two positions [9].
- **Uniform Crossover:** For each gene position, randomly decide whether to exchange the gene at that position [10].

Mutation: Perform mutation operations on the offspring individuals to introduce new genetic features. Common mutation methods include:

- **Bit-Flip Mutation:** For binary encoding, randomly select some positions and flip the bits (change 0 to 1 or 1 to 0) [11].
- **Swap Mutation:** Randomly select two positions and swap the genes at these positions [12].
- **Gaussian Mutation:** For real-valued encoding, add a random value drawn from a Gaussian distribution to the original gene value [13].

Update population: Combine the offspring individuals with the original population and select a fixed number of individuals based on their fitness values to form the new population for the next iteration.

Termination condition: Repeat steps 2-6 until a termination condition is met (e.g., reaching the maximum number of iterations, finding a satisfactory solution, etc.).

B. Optimize DNN with GA

There are many different directions or ways to optimize deep neural networks through GA, because deep neural networks have various parameters at different levels that need to be predefined. GA are applied in various optimization scenarios of automated DNN construction, such as learning model parameters, designing model architectures, and optimizing model learners.

For **learning model parameters**, GAs use direct encoding or indirect encoding (binary encoding to bit strings) way to encode model parameter (e.g., weights and biases) to the chromosome (the genetic representation of a solution). However, since the search cost of GA is too large compared to the gradient descent search method, often use hybrid approaches that combined with gradient-based methods to optimize DNNs to help gradient-based methods jump out of local optimal solutions. Such as using the EA and the gradient-based method alternatively where the output of one method servers as the start point for another, and iterate this procedure until some stopping criteria are met [14].

To use GAs for **DNN architecture optimization**, the architecture must be encoded into a chromosome that GA can manipulate. This can involve:

- **Fixed Topological Patterns:** A fixed representation where the architecture is defined by a sequence of layers

and their respective parameters. For example, a vector can encode the type of layer (convolutional, pooling, fully connected) and its parameters (number of neurons, kernel size, etc.).

- **Nonfixed Topological Patterns:** More complex representations that also include the connections between layers. This may use data structures like adjacency matrices or lists [15] to represent which layers are connected.

GAs employ several search paradigms to explore the space of possible architectures:

- **Basic Search Paradigm:** Following the standard GA process—initializing a population of architectures, evaluating them, and using genetic operators to create new generations until a termination condition is met.
- **Incremental Search Paradigm:** Architectures are evolved by incrementally adding or altering components as the generations progress. This can reduce computational costs as the search space is explored gradually.
- **Cooperative Co-Evolution Paradigm:** Decomposes the architecture into subcomponents that are evolved separately but in a manner that ensures they work well together. This can be seen as a form of divide-and-conquer approach [16].
- **Hypernet-Based Search Paradigm:** Utilizes a hypernetwork that embodies a superset of architectures. Candidate architectures are sampled from this hypernet and inherit its trained weights. This approach can substantially speed up the evaluation of architectures.

DNN architecture optimization has unique requirements, GAs often use customized genetic operators:

- **Customized Recombination (Crossover):** Tailored to handle variable-length chromosomes, ensuring that offspring DNNs are valid architectures.
- **Customized Mutation:** In addition to modifying parameters, mutation operators may add or remove layers or alter connections, facilitating architectural exploration.
- **Repair Operators:** Used to correct any invalid architectures that may arise due to crossover or mutation, ensuring that all architectures in the population are feasible [17].
- **Inheritance Operators:** Enable the transfer of weights from parent architectures to offspring, which can reduce the need for full retraining and thus accelerate the optimization process.

Moreover, GAs are not limited to optimizing a single objective like accuracy. They are also used to find architectures that balance multiple objectives, which is essential when designing DNNs for real-world applications where trade-offs must be made between accuracy, computational cost, memory usage, and more [18].

To *optimize model learners* by GAs, it usually involves searching for the best model parameter learner, which includes finding both the most suitable algorithm type (e.g., SGD, Adam, RMSprop) and its hyperparameters (e.g., learning rate, momentum).

III. METHODOLOGY

In this study, we propose genetic algorithm (GA) approach for optimizing the hyperparameters of a classifier. Because there are some differences of hyperparameters setting between traditional machine learning methods and deep neural networks, so we provided two versions GAs.

A. GA-Searching best hyperparameters of traditional machine learning methods

GA operates on a population of candidate solutions, referred to as chromosomes, and iteratively evolves them through selection, crossover, and mutation operations to find the optimal solution.

The proposed GA takes the following inputs: the classifier to be optimized, the ranges of hyperparameters to be tuned, the population size, the number of generations, the diversity threshold *di_threshold*, the tournament size *k*, and the crossover rate *crossrate*. The GA begins by initializing a population of random chromosomes, where each chromosome represents a set of hyperparameter key-value pairs within the specified ranges. For different datatype of hyperparameter (categorical, int and float), we use different rules for initialize.

The fitness of each chromosome is evaluated using F1-macro. Clones the classifier, sets its hyperparameters according to the chromosome, and computes the mean F1-macro score using 5-fold cross-validation on the training data. The fitness score reflects the performance of the classifier with the given hyperparameter configuration. And the F1-macro score can be formulated as:

$$\begin{aligned} \text{F1-macro} &= 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \\ \text{precision} &= \frac{\text{true positives}}{\text{true positives} + \text{false positives}} \\ \text{recall} &= \frac{\text{true positives}}{\text{true positives} + \text{false negatives}} \end{aligned}$$

To maintain population diversity, we introduce a diversity score calculation based on the pairwise differences between chromosomes. The `calculate_diversity` method computes the diversity score for each chromosome by summing the differences in hyperparameter values compared to other chromosomes in the population. if parameter type is numeric, we use absolute difference to identify the difference; if it is categorical, we use 0 and 1 to identify whether the two are the same.

The selection process employs tournament selection, where *k* chromosomes are randomly selected from the population, and the one with the highest fitness score is chosen as the winner. This process is repeated until a new population of the same size is formed.

Crossover is performed using two-point crossover, where two parent chromosomes are selected randomly, and their genetic material is exchanged between two randomly selected

crossover points. The crossover operation is applied with a probability specified by the *crossrate*.

Mutation is implemented as an adaptive mutation strategy [19], where the mutation rate is adjusted based on the population diversity. If the average diversity score falls below the *di_threshold*, a higher mutation rate is used to introduce more exploration. Otherwise, a lower mutation rate is used to fine-tune the solutions. Mutation randomly alters the values of hyperparameters within their respective ranges.

The GA iterates for a specified number of generations, performing selection, crossover, and mutation in each generation. The best individual, i.e., the chromosome with the highest fitness score, is returned as the optimal hyperparameter configuration for the classifier.

The proposed GA-based hyperparameter optimization approach offers several advantages. It efficiently explores the hyperparameter space by maintaining a diverse population and adapting the mutation rate based on population diversity. The tournament selection and two-point crossover operations promote the survival and propagation of promising hyperparameter configurations. To control diversity threshold of mutation, *k* of tournament selection and *crossrate*, we can simply balance GA's exploration and exploitation, allowing the GA to escape local optima and fine-tune solutions. For example, if we choose higher *k* and lower *di_threshold*, the model will more focus on exploitation; On the contrary, lower *k* and higher *di_threshold* will more focus on exploration. A general framework is described in Algorithm 1.

B. GA-Searching best hyperparameters and architecture of Multi-layer Perceptron

In previous works, most of them use GA optimizes MLP for optimizing inner parameters (weights and biases of network) to replace **gradient descent way** [20], [21]. This way can help neural network jump out of local optima during training, however, compared with gradient descent method, there is no clear optimization path for updating weight values, making convergence difficult during training. Besides, there are many methods that have been proven effective in practical situations to help gradient descent optimization jump out of local optima at present, such as Adam [22], Gradient Noise [23].

In numerous research, it has been demonstrated that the **structure** of neural networks (e.g., the number of hidden layers and the number of neurons in each hidden layer) and associated hyperparameters (such as activation functions, initial learning rates, etc.) significantly affect model performance. However, manually adjusting these hyperparameters is a time-consuming and error prone process, as the combination space of hyperparameters is very large and there may be complex interaction relationships between different hyperparameters. Compared with manual parameter tuning or grid search methods, GA as a heuristic optimization algorithm can explore the hyperparameter space more intelligently and efficiently, especially in situations with a large number of hyperparameters and complex interactions. GA can find near

Algorithm 1: Genetic Algorithm for Hyperparameter Optimization of traditional machine learning methods

Input : Classifier, parameter ranges, population size, number of generations, diversity threshold, tournament size *k*, crossover rate

Output: Best hyperparameter set

```
// Initialize population
// Main loop
1 for generation  $\leftarrow 1$  to num_generations do
    // Fitness evaluation
    // Diversity calculation
    // Tournament selection
2     new_population  $\leftarrow \emptyset$ ;
3     while size(new_population) < population_size
4         do
5             Randomly select k chromosomes;
6             Select the chromosome with the highest fitness
7             score;
8             Add the selected chromosome to
9             new_population;
10        end
11    population  $\leftarrow$  new_population
12    // Two-point crossover base on
13    cross_rate
14    // Adaptive mutation
15    avg_diversity  $\leftarrow$  mean(diversity_scores);
16    if avg_diversity > diversity_threshold then
17        | mutation_rate  $\leftarrow$  low_mutation_rate;
18    else
19        | mutation_rate  $\leftarrow$  high_mutation_rate;
20    end
21    mutate base on mutation_rate
22 end
```

optimal hyperparameter combinations in a reasonable amount of time, thereby improving the performance of MLP.

In some previous studies, when optimizing MLP structures using GA, the number of hidden layers was typically fixed. [24], [25] Subsequently, the number of neurons in each hidden layer was combined with other hyperparameters to form chromosomes for the evolution of hyperparameters. We utilized specialized chromosome initialization, crossover, and mutation operators, enabling changes to both the number of hidden layers and neurons during GA's search for MLP structures, thereby further expanding the search space. Additionally, we introduced elitism, which enhances model exploitation capability while ensuring diversity.

A general framework is described in Algorithm 2. And a example diagram of crossover and mutation is described in Figure 1.

In order to further elaborate on our method, we will provide a detailed introduction to the key components of GA in optimizing MLP structure and hyperparameters.

- **Chromosome coding:** In our method, each chromosome represents an MLP network structure and related hyperparameter combinations. Chromosomes contain the following information: number of hidden layers, number of neurons in each hidden layer, activation function type, solver type, regularization parameters (such as L2 regularization) alpha Value and initial learning rate. This encoding method allows GA to optimize both network structure and hyperparameters during the search process.
- **Initialize population:** We have designed a special initialization mechanism to ensure the diversity of the initial population. For each chromosome, we randomly generate the number of hidden layers (ranging from 1 to 5), and then randomly assign the number of neurons within a given range to each hidden layer. Other hyperparameters are also randomly initialized within a predefined range. This initialization method generates diverse MLP structures and hyperparameter combinations, providing a good starting point for the subsequent evolution process.
- **Fitness evaluation:** Similar to the previous algorithm, a 3-fold cross validation method is used, and the average score of macro-F1 is used as the fitness value. This evaluation method provides a reliable estimate of the model's generalization ability, which helps to select individuals with excellent performance.
- **Selection operation:** We adopt a roulette wheel selection method with elitism. Firstly, we sort the population based on fitness values and directly replicate the best performing individuals (elites) to the next generation based on *elitism_rate*. Then, based on the proportion of fitness values, we select parents from non elite individuals through roulette wheel selection. This selection mechanism not only ensures the survival of excellent individuals, but also gives the weak a certain opportunity to maintain population diversity.
- **Cross operation:** We have designed specialized cross operators to adapt to the optimization of MLP structure and hyperparameters, which separate the structural hyperparameters from other hyperparameters. For two parental chromosomes, we perform crossover operations with a certain probability *cross_rate*. If the crossover condition is met, we randomly select a crossover point within the smaller number of hidden layers in the two parent classes, and then exchange the hidden layer structure of the parent chromosome after that point. Other hyperparameters are inherited from one of the parent chromosomes with a 50
- **Mutation operation:** Similarly, we have introduced specialized mutation operators to separate structural hyperparameters from other hyperparameters for mutation. For each non elite chromosome, we mutate it with a certain probability *mutation_rate*. For the hidden layer structure, we randomly choose to add or remove a hidden layer with a random number of neurons, or modify the number of neurons in a certain layer. For other hyperparameters, we randomly assign new values within a predefined range.

Similar to selection operations, crossover and mutation are executed within non-elite populations to safeguard the algorithm's exploitation capacity. The proposed GA-based hyperparameter optimization approach aims to efficiently search the hyperparameter space and find a near-optimal configuration for the MLP classifier. By leveraging the principles of natural selection and evolution, the GA explores various combinations of hyperparameters and evolves towards better solutions over successive generations. Moreover, to control and set different *elitism_rate*, *cross_rate*, *mutation_rate*, we enable to simply balance exploration and exploitation.

Algorithm 2: GA for MLP Structure and Hyperparameter Optimization

Input : Classifier, parameter ranges, population size, generations, elitism rate, crossover rate, mutation rate, dataset (X, y)

Output: Best MLP structure and hyperparameters

```

1 Initialize population with random MLP structures and
  hyperparameters;
2 for generation  $\leftarrow 1$  to generations do
3   for chromosome in population do
4     Calculate fitness of chromosome using 3-fold
      cross-validation on  $(X, y)$ ;
5   end
6   Select elites based on fitness scores;
7   Select parents from non-elites using roulette wheel
    selection;
8   for  $i \leftarrow 1$  to population size - elites do
9     if random() < crossover rate then
10      Select two parents from population;
11      Perform crossover on hidden layer sizes
        and hyperparameters;
12      Add offspring to new population;
13    end
14    else
15      Add one parent to new population;
16    end
17  end
18  for chromosome in new population (excluding
    elites) do
19    if random() < mutation rate then
20      Mutate hidden layer sizes (add, remove, or
        modify layers);
21      Mutate other hyperparameters;
22    end
23  end
24  Update population with new population and elites;
25 end
26 return Best chromosome (MLP structure and
  hyperparameters)

```

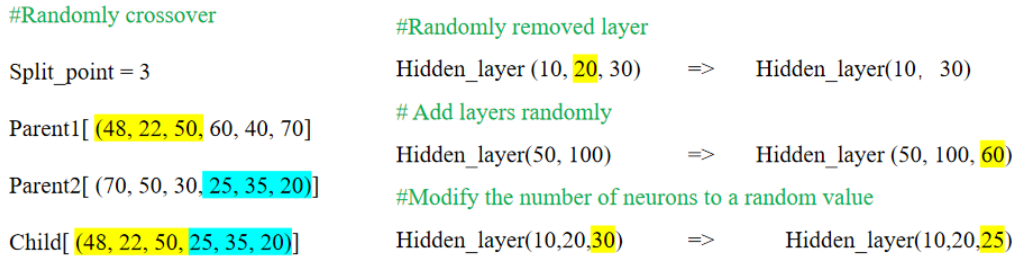


Fig. 1. Example diagram of crossover and mutation of MLP architecture.

IV. EXPERIMENT

A. Dataset

The dataset we use the **"Heart Disease"** dataset in **UCI Machine Learning Repository, "Cleveland" database** [4]. In the realm of machine learning research, the Cleveland database stands out by providing a subset of 14 out of the 76 available attributes for analysis, despite the full dataset's extensive attribute count. To date, this subset has been the primary focus of scholarly investigations. The Cleveland database uniquely facilitates studies on heart disease prediction by incorporating a "goal" field, which quantifies the presence of heart disease on a scale from 0 (indicating no presence) to 4. Research efforts using this database have predominantly aimed at binary classification tasks, distinguishing between the absence of heart disease (value 0) and its presence (values 1 through 4). This approach simplifies the experimental design and focuses on the critical binary distinction crucial for initial diagnostic evaluations.

Continuous Variables:

- **age** - Age, usually in years, can be taken to a decimal place.
- **trestbps** - Resting blood pressure, measured in millimeters of mercury (mmHg).
- **CHOL** - Serum cholesterol levels, usually measured in milligrams per deciliter (mg/dl), are a continuous variable.
- **Thalach** - Maximum heart rate, measured in beatings per minute, can be taken as any value within a certain range.
- **oldpeak** - The peak ECG after exercise, although it may be an integer, but it is also a continuous variable if decimals are allowed.
- **slope** - The slope of the ST segment, although it is usually classified as 0 (descending), 1 (flat) or 2 (rising).

Categorical Variables:

- **sex** - Gender, with possible values being 1 (usually for male) and 0 (usually for female).
- **CP** - Type of chest pain, with possible values ranging from 0 to 3 or higher depending on the frequency and duration of chest pain.
- **FBS** - Fasting blood glucose level, a binary variable with possible values of 1 (indicating diabetes or high fasting

blood glucose) and 0 (indicating no diabetes or normal fasting blood glucose).

- **restecg** - Resting ECG results, with possible values of 0 (normal) or 1 (abnormal).
- **exang** - Exercise-induced angina, a binary variable with possible values of 1 (yes) and 0 (no).
- **ca** - Coronary artery disease, a binary variable with possible values of 1 (with coronary artery disease) and 0 (without coronary artery disease).
- **thal** - Cardiac response after exercise, with possible values including fixed numbers (e.g., 1, 2, 3, etc.).

In order to improve the performance of the binary classification model, we applied MinMaxScaler to scale the continuous variables in the data. The formula can be described as:

$$x_{scaled} = \frac{x - \min(x)}{\max(x) - \min(x)} \cdot (b - a) + a$$

where:

- x is the original feature value
- $\min(x)$ is the minimum value of feature x
- $\max(x)$ is the maximum value of feature x
- a is the lower bound of the scaled range, typically 0
- b is the upper bound of the scaled range, typically 1

In practice, MinMaxScaler calculates the minimum and maximum values for each feature in the dataset separately, and then applies the above formula to scale all samples of each feature.

The scaled feature value x_{scaled} will fall within the range $[a, b]$. By scaling in this manner, MinMaxScaler transforms the feature values to their relative magnitudes within the specified range while preserving the distribution characteristics of the original data.

Finally, we randomly split the dataset with a fix seed (to ensure the reproducibility of the experiment), use 80% of data as training data, 20% as test data.

B. Hyperparameter Setting

In this study, we investigated the effectiveness of hyperparameter optimization using genetic algorithms (GA) for various machine learning models, including Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, and Multilayer Perceptron (MLP). We compared the performance of the default models, models optimized

using grid search, and models optimized using GA, evaluating their performance using the F1 macro score. Specifically, we use `scikit-learn` [26] python package to implement our experiments.

Logistic Regression is a statistical method that models the probability of a binary outcome based on one or more predictor variables.

The hyperparameters of Logistic Regression searched by GAs is:

- **solver**: Specifies the algorithm used for optimization to minimize the loss function. Different algorithms perform differently on different types of datasets, so the selection of a solver needs to be balanced based on the dataset size and characteristics.
The available types of 'solver' are ['lbfgs', 'liblinear', 'newton-cg', 'newton-cholesky', 'saga']
- **C**: The inverse of the regularization strength and controls the strength of regularization. The C value determines the strength of regularization. A smaller C value helps prevent overfitting, while a larger C value may lead to overfitting of the training data.
The appropriate range of 'C' is (0.1, 10.0)
- **penalty**: The type of regularization. The choice of penalty affects the regularization method. Different regularization types have different penalization methods for model parameters, which can result in different feature selection and model sparsity.
The available types of 'penalty' are ['l1', 'l2', 'elasticnet', None]

And for parameters of Genetic Algorithms, we use *population_size*=50, *generations*=100, and *di_threshold*=0.3, *k*=3, *crossrate*=0.7 as default setting.

To compare with GridSearch way, set Grid_parameters is:

```
{
    'penalty': ['l1', 'l2'],
    'C': [0.1, 1.0, 5.0, 10.0],
    'solver': ['liblinear', 'saga']
}
```

Random Forest is an ensemble learning method that constructs a multitude of decision trees and outputs the class that is the mode of the classes of the individual trees.

The hyperparameters of Random Forest searched by GAs are:

- **n_estimators**: The number of trees in the forest. A larger number of trees can improve performance, but also increases computation time.
The appropriate range of 'n_estimators' is (10, 200)
- **max_depth**: The maximum depth of each tree. Deeper trees can capture more complex relationships, but may also lead to overfitting.
The appropriate range of 'max_depth' is (5, 30)
- **min_samples_split**: The minimum number of samples required to split an internal node. Higher values can prevent overfitting, but may also lead to underfitting.

The appropriate range of 'min_samples_split' is (2, 10)

- **max_features**: The number of features to consider when looking for the best split. Selecting a subset of features at each split can reduce overfitting and improve computational efficiency.
The available options for 'max_features' are ['sqrt', 'log2', None]
- **criterion**: The function to measure the quality of a split. Different criteria can be used to evaluate the quality of splits and influence the tree structure.
The available options for 'criterion' are ['gini', 'entropy', 'log_loss']

For the parameters of Genetic Algorithms, we use And for parameters of Genetic Algorithms, we use *population_size*=50, *generations*=100, and *di_threshold*=0.3, *k*=3, *crossrate*=0.7 as default setting.

To compare with the GridSearch approach, the Grid_parameters are set as:

```
{
    'penalty': ['l1', 'l2'],
    'C': [0.1, 1.0, 5.0, 10.0],
    'solver': ['liblinear', 'saga']
}
```

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression tasks. It aims to find the optimal hyperplane that maximally separates different classes.

The hyperparameters of SVM searched by the Genetic Algorithm are:

- **C**: The regularization parameter that trades off misclassification of training examples against simplicity of the decision surface. A low C makes the decision surface smooth, while a high C aims at classifying all training examples correctly.
The appropriate range of 'C' is (0.1, 10.0)
- **kernel**: The kernel function used to transform the input data into a higher-dimensional space. Different kernels capture different types of decision boundaries.
The available options for 'kernel' are ['linear', 'poly', 'rbf', 'sigmoid']
- **gamma**: The kernel coefficient for 'rbf', 'poly', and 'sigmoid' kernels. It defines how far the influence of a single training example reaches.
The appropriate range of 'gamma' is (0.01, 10.0)

For the Genetic Algorithm parameters, we use *population_size*=50, *generations*=100, *crossrate*=0.7 as default setting. Further more, use higher *di_threshold*=0.2, *k*=5 to enhance exploitation ability of algorithm.

To compare with the GridSearch approach, the Grid_parameters are set as:

```
{
    'C': [0.01, 0.1, 1, 5, 10],
    'kernel': ['poly', 'rbf'],
    'gamma': [0.01, 0.1, 1, 10]
```

}

XGBoost (Extreme Gradient Boosting) is an ensemble learning algorithm that uses gradient boosting to build a model. It is widely used for classification and regression tasks due to its high performance and scalability.

The hyperparameters of XGBoost searched by the Genetic Algorithm are:

- **max_depth**: The maximum depth of a tree. Increasing this value will make the model more complex and more likely to overfit.
The appropriate range of 'max_depth' is (2, 10).
- **learning_rate**: The step size shrinkage used in update to prevents overfitting. After each boosting step, we can directly get the weights of new features, and learning_rate shrinks the feature weights to make the boosting process more conservative.
The appropriate range of 'learning_rate' is (0.01, 0.3).
- **subsample**: The subsample ratio of the training instances. Setting it to 0.5 means that XGBoost would randomly sample half of the training data prior to growing trees, which can prevent overfitting.
The appropriate range of 'subsample' is (0.5, 1.0).
- **colsample_bytree**: The subsample ratio of columns when constructing each tree. Subsampling occurs once for every tree constructed.
The appropriate range of 'colsample_bytree' is (0.5, 1.0).
- **gamma**: The minimum loss reduction required to make a further partition on a leaf node of the tree. The larger gamma is, the more conservative the algorithm will be.
The appropriate range of 'gamma' is (0, 0.5).

For the Genetic Algorithm parameters, *population_size*=50, *generations*=100 and *di_threshold*=0.3, *k*=3, *crossrate*=0.7 are used as the default settings.

To compare with the GridSearch approach, the Grid_parameters are set as:

```
{
  'max_depth': [3, 5, 8, 10, 20],
  'learning_rate': [0.015, 0.1, 0.2, 0.3],
  'subsample': [0.6, 0.8, 1.0],
  'colsample_bytree': [0.6, 0.8, 1.0],
  'gamma': [0, 0.1, 0.2]
}
```

Multi-layer Perceptron (MLP) is a class of feedforward artificial neural network. It consists of at least three layers of nodes: an input layer, a hidden layer, and an output layer. Except for the input nodes, each node is a neuron that uses a nonlinear activation function.

The hyperparameters of MLP searched by the Genetic Algorithm are:

- **activation**: The activation function for the hidden layer. The available options for 'activation' are ['tanh', 'relu'].
- **solver**: The solver for weight optimization. 'adam' refers to a stochastic gradient-based optimizer, and 'sgd' refers to stochastic gradient descent.

The available options for 'solver' are ['adam', 'sgd'].

- **alpha**: The L2 penalty (regularization term) parameter. The appropriate range of 'alpha' is (0.0001, 0.05).
- **learning_rate_init**: The initial learning rate used. It controls the step size when updating the weights. The appropriate range of 'learning_rate_init' is (0.001, 0.01).

Specially, GA-MLP will randomly generate the number of hidden layers (ranging from 1 to 5), and then randomly assign the number of neurons within a given range (set 5 to 100) to each hidden layer for each individuals.

For the Genetic Algorithm parameters, *population_size*=50, *generations*=50 are used. *elitism_rate*=0.1, *crossover_rate*=0.7, *mutation_rate*=0.2 as the default settings.

To compare with the GridSearch approach, the Grid_parameters are set as:

```
{
  'hidden_layer_sizes': [(50,), (100,), (50, 50)],
  'activation': ['tanh', 'relu'],
  'solver': ['sgd', 'adam'],
  'alpha': [0.001, 0.05],
  'learning_rate_init': [0.001, 0.01]
}
```

In the grid search, an hyperparameter 'hidden_layer_sizes' defines the number of neurons in the hidden layers. The options [(50,), (100,), (50, 50)] represent one hidden layer with 50 neurons, one hidden layer with 100 neurons, and two hidden layers with 50 neurons each, respectively.

V. RESULTS

After experiments, the final results is illustrated in Table I.

TABLE I
F1 MACRO(%) SCORES FOR DIFFERENT CLASSIFIERS

	Logistic Regression	Random Forest	SVM	XGBoost	MLP
Baseline	85.18	83.6	83.6	80.3	78
Grid Search	85.18	85.23	78.66	81.9	86
*GA Search	86.88	85.18	85.18	83.6	87

In this study, we investigated the effectiveness of hyperparameter optimization using genetic algorithms (GA) for various machine learning models, including Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, and Multilayer Perceptron (MLP). We compared the performance of the default models, models optimized using grid search, and models optimized using GA, evaluating their performance using the F1 macro score.

The results demonstrate that hyperparameter optimization, both through grid search and GA, generally improves the performance of the models compared to their default configurations. GA optimization consistently outperformed or matched the performance of grid search for all the models tested, highlighting its effectiveness in finding optimal hyperparameter combinations.

For **Logistic Regression**, the best individual found by GA used the Stochastic Average Gradient Descent (SAGA) solver, a C value of 7.23, and no penalty. This configuration achieved an F1 macro score of 86.88%, surpassing both the default and grid search optimized models.

The best individual for **Random Forest**, discovered by GA, consisted of 92 estimators, a maximum depth of 13, a minimum of 3 samples required to split an internal node, the 'log2' criterion for the number of features to consider, and the 'log_loss' criterion for measuring the quality of a split. This configuration achieved an F1 macro score of 85.18%, outperforming the default model and matching the grid search optimized model.

For **SVM**, the best individual found by GA used an RBF kernel with a C value of 1.085 and a gamma value of 4.909. This configuration achieved an F1 macro score of 85.18%, surpassing both the default and grid search optimized models.

The best individual for **XGBoost**, discovered by GA, had a maximum depth of 6, a learning rate of 0.089, a subsample ratio of 0.779, a colsample_bytree ratio of 0.92, and a gamma value of 0.038. This configuration achieved an F1 macro score of 83.6%, outperforming both the default and grid search optimized models.

The optimal set of parameters for the **Multilayer Perceptron (MLP)** as determined by Genetic Algorithms includes a configuration of hidden layers with sizes 49 and 15, the 'relu' activation function, the 'sgd' solver for weight optimization, a regularization term (alpha) of approximately 0.034, and an initial learning rate of about 0.009. And the best individual found by GA achieved an F1 macro score of 87%, surpassing both the default and grid search optimized models, and this score is the highest among all the experiments.

In conclusion, our study demonstrates the potential of genetic algorithms for hyperparameter optimization in machine learning models. GA consistently found hyperparameter configurations that improved model performance compared to default settings and grid search optimization. These findings suggest that GA is a promising approach for automating the hyperparameter tuning process and can lead to significant improvements in model performance.

VI. CONCLUSION

In this study, we explored the application of genetic algorithms (GA) for hyperparameter optimization in various machine learning models and compared their performance with grid search and default model settings. Our results demonstrate that GA is an effective and efficient approach for finding near-optimal hyperparameter configurations, consistently outperforming or matching the performance of grid search and default settings across all the models tested.

The success of GA in hyperparameter optimization can be attributed to its unique characteristics. GA guides the search process using fitness scores, which gradually evolve the population towards hyperparameter combinations with better performance. By retaining well-performing combinations

and giving them more opportunities to participate in subsequent evolution, GA propagates and combines good "genes" within the population to produce better offspring. Furthermore, through crossover and mutation operations, GA can generate new hyperparameter combinations, allowing it to explore a wider search space and discover potentially unexpected, well-performing solutions.

In contrast, grid search is a more mechanical approach that exhaustively evaluates all predefined hyperparameter combinations without utilizing existing information to guide the search. Each combination is evaluated independently, and grid search lacks the ability to innovate or generate new combinations. As the range of hyperparameter values increases, the computational cost of grid search grows exponentially, quickly becoming impractical for large search spaces.

Our findings are consistent with previous research that highlights the advantages of GA for hyperparameter optimization. For example, *Xiao et al. (2020)* [27] demonstrated that GA could effectively find the hyperparameters settings for deep convolutional neural networks. Similarly, *Ahmad et al. (2018)* [28] showed that GA can produce a better hyper parameter for machine learning than Grid Search in a reasonable amount of time.

The current study contributes to the growing body of literature on the application of evolutionary algorithms in machine learning. By comparing GA with grid search across a range of models and datasets, we provide further evidence of the effectiveness and versatility of GA for hyperparameter optimization. Our results suggest that GA is a promising approach for automating the hyperparameter tuning process, offering the potential to significantly improve model performance while reducing the computational burden associated with exhaustive search methods like grid search.

In conclusion, genetic algorithms have shown to be a powerful and efficient tool for hyperparameter optimization in machine learning models. By leveraging the principles of natural selection and evolution, GA can effectively navigate complex search spaces and find near-optimal hyperparameter configurations in an acceptable time. As the complexity of machine learning models continues to grow, the use of evolutionary algorithms like GA for hyperparameter optimization is likely to become increasingly important in the field of machine learning.

VII. FUTURE WORK

Currently, there are more methods available for balancing the development and exploration of genetic algorithms. Clustering methods mainly achieve a balance between population diversity and selection pressure by dividing the population into multiple subpopulations (also known as 'niches' or 'species') and performing selection and genetic operations within the subpopulations, thus avoiding premature convergence. These methods are known as niching methods. They can be mainly divided into the following categories:

1) Fitness-based clustering methods

- a) Fitness sharing: Dividing an individual's fitness by the number of other individuals in its niche, thereby reducing the fitness of individuals in crowded niches and maintaining balance between niches [29].
- b) Clearing: Similar to fitness sharing, but only retaining the top k individuals in each niche and setting the fitness of other individuals to zero. This can better preserve local optima [30].
- c) Modified clearing: An improvement to clearing that avoids setting the parameter k and adaptively adjusts the niche radius [31].
- d) Clustering: First clustering the population based on fitness, then performing selection within each cluster, ensuring that each cluster has a certain opportunity to reproduce [32].

2) Replacement-based clustering methods

- a) Deterministic crowding: Each offspring individual is compared with its most similar parent individual, and the superior one is retained. This forms local competition, and new individuals replace more similar parent individuals [33].
- b) Probabilistic crowding: Similar to deterministic crowding, but introduces a certain probability to accept inferior solutions and update similar parents [34].
- c) Restricted tournament selection: Tournament selection is performed within local populations, and the winning individual replaces the most similar individual in that local population [35].

3) Protection-based clustering methods

- a) Species conserving GA: Divides individuals into multiple species based on similarity and specifies a minimum number of individuals for each species. Individuals are only allowed to be replaced when there are more than the minimum number within a species [36].

4) Hybrid clustering methods

Combine multiple clustering methods, such as first using fitness sharing to divide niches and then using crowding selection within niches. The combination of different methods can leverage their respective advantages [37].

In the future, we hope to try using these methods to improve genetic algorithms that simply control selection pressure to balance development and exploration, and study their effectiveness for searching neural network structures.

REFERENCES

- [1] M. Črepinšek, S.-H. Liu, and M. Mernik, "Exploration and exploitation in evolutionary algorithms: A survey," *ACM Comput. Surv.*, vol. 45, jul 2013.
- [2] M. Potter, A. Y. Yıldız, N. M. Prabhu, and C. Gordon, "Distributed genetic algorithm for feature selection," *arXiv preprint arXiv:2401.10846*, 2024.
- [3] X. Zhou, A. K. Qin, M. Gong, and K. C. Tan, "A survey on evolutionary construction of deep neural networks," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 894–912, 2021.
- [4] S. W. P. M. Janosi, Andras and R. Detrano, "Heart Disease." UCI Machine Learning Repository, 1988. DOI: <https://doi.org/10.24432/C52P4X>.
- [5] D. E. Goldberg and K. Deb, "A comparative analysis of selection schemes used in genetic algorithms," in *Foundations of genetic algorithms*, vol. 1, pp. 69–93, Elsevier, 1991.
- [6] B. L. Miller, D. E. Goldberg, et al., "Genetic algorithms, tournament selection, and the effects of noise," *Complex systems*, vol. 9, no. 3, pp. 193–212, 1995.
- [7] J. E. Baker, "Adaptive selection methods for genetic algorithms," in *Proceedings of the first international conference on genetic algorithms and their applications*, pp. 101–106, Psychology Press, 2014.
- [8] J. H. Holland, *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press, 1992.
- [9] A. de JONG Kenneth, "Analysis of the behavior of a class of genetic adaptive systems," *Technical Report No. 185, Department of Computer and Communication Sciences, University of Michigan*, 1975.
- [10] G. Syswerda et al., "Uniform crossover in genetic algorithms," in *ICGA*, vol. 3, pp. 2–9, 1989.
- [11] H. Muhlenbein, "How genetic algorithms really work: I. mutation and hillclimbing," in *Proc. 2nd Int. Conf. on Parallel Problem Solving from Nature*, 1992, Elsevier, 1992.
- [12] W. Banzhaf, "The "molecular" traveling salesman," *Biological Cybernetics*, vol. 64, no. 1, pp. 7–14, 1990.
- [13] R. Hinterding, Z. Michalewicz, and T. C. Peachey, "Self-adaptive genetic algorithm for numeric functions," in *Parallel Problem Solving from Nature—PPSN IV: International Conference on Evolutionary Computation—The 4th International Conference on Parallel Problem Solving from Nature Berlin, Germany, September 22–26, 1996 Proceedings 4*, pp. 420–429, Springer, 1996.
- [14] M. Gong, J. Liu, A. K. Qin, K. Zhao, and K. C. Tan, "Evolving deep neural networks via cooperative coevolution with backpropagation," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 420–434, 2020.
- [15] M. Suganuma, M. Kobayashi, S. Shirakawa, and T. Nagao, "Evolution of deep convolutional neural networks using cartesian genetic programming," *Evolutionary computation*, vol. 28, no. 1, pp. 141–163, 2020.
- [16] J. Liang, E. Meyerson, and R. Miikkulainen, "Evolutionary architecture search for deep multitask networks," in *Proceedings of the genetic and evolutionary computation conference*, pp. 466–473, 2018.
- [17] F. Assunção, N. Lourenço, P. Machado, and B. Ribeiro, "Denser: deep evolutionary network structured representation," *Genetic Programming and Evolvable Machines*, vol. 20, pp. 5–35, 2019.
- [18] Z. Lu, G. Sreeksumar, E. Goodman, W. Banzhaf, K. Deb, and V. N. Boddeti, "Neural architecture transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 9, pp. 2971–2989, 2021.
- [19] M. Srinivas and L. M. Patnaik, "Adaptive probabilities of crossover and mutation in genetic algorithms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 24, no. 4, pp. 656–667, 1994.
- [20] R. Zhang, Z. Zhang, D. Wang, and M. Du, "Financial distress prediction with a novel diversity-considered ga-mlp ensemble algorithm," *Neural Processing Letters*, vol. 54, no. 2, pp. 1175–1194, 2022.
- [21] P. Bansal, R. Lamba, V. Jain, T. Jain, S. Shokeen, S. Kumar, P. K. Singh, B. Khan, et al., "Gga-mlp: A greedy genetic algorithm to optimize weights and biases in multilayer perceptron," *Contrast Media & Molecular Imaging*, vol. 2022, 2022.
- [22] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of machine learning research*, vol. 12, no. 7, 2011.
- [23] A. Neelakantan, L. Vilnis, Q. V. Le, I. Sutskever, L. Kaiser, K. Kurach, and J. Martens, "Adding gradient noise improves learning for very deep networks," *arXiv preprint arXiv:1511.06807*, 2015.
- [24] O. Y. Dwekat and S. S. Lam, "Optimized design of hybrid genetic algorithm with multilayer perceptron to predict patients with diabetes," *Soft Computing*, vol. 27, no. 10, pp. 6205–6222, 2023.
- [25] O. Y. Dwekat and S. S. Lam, "Cervical cancer diagnosis using an integrated system of principal component analysis, genetic algorithm, and multilayer perceptron," *Healthcare*, vol. 10, no. 10, 2022.
- [26] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

- [27] X. Xiao, M. Yan, S. Basodi, C. Ji, and Y. Pan, "Efficient hyperparameter optimization in deep learning using a variable length genetic algorithm," *arXiv preprint arXiv:2006.12703*, 2020.
- [28] A. S. Wicaksono and A. A. Supianto, "Hyper parameter optimization using genetic algorithm on machine learning methods for online news popularity prediction," *International Journal of Advanced Computer Science and Applications*, vol. 9, no. 12, 2018.
- [29] D. E. Goldberg and J. Richardson, "Genetic algorithms with sharing for multimodal function optimization," in *Genetic algorithms and their applications: Proceedings of the Second International Conference on Genetic Algorithms*, pp. 41–49, Lawrence Erlbaum Hillsdale, NJ, 1987.
- [30] A. Petrowski, "A clearing procedure as a niching method for genetic algorithms," in *Proceedings of IEEE international conference on evolutionary computation*, pp. 798–803, IEEE, 1996.
- [31] G. Singh and K. Deb, "Comparison of multi-modal optimization algorithms based on evolutionary algorithms," in *Proceedings of the 8th annual conference on Genetic and evolutionary computation*, pp. 1305–1312, 2006.
- [32] X. Yin and N. Gernay, "A fast genetic algorithm with sharing scheme using cluster analysis methods in multimodal function optimization," in *International conference on artificial neural nets and genetic algorithms*, pp. 450–457, Springer, 1993.
- [33] S. W. Mahfoud, "Crowding and preselection revisited," Tech. Rep. 51, Urbana, 1992.
- [34] O. J. Mengshoel and D. E. Goldberg, "Probabilistic crowding: Deterministic crowding with probabilistic replacement," in *GECCO*, pp. 409–416, 1999.
- [35] G. R. Harik, "Finding multimodal solutions using restricted tournament selection," in *ICGA*, pp. 24–31, 1995.
- [36] J.-P. Li, M. E. Balazs, G. T. Parks, and P. J. Clarkson, "A species conserving genetic algorithm for multimodal function optimization," *Evolutionary computation*, vol. 10, no. 3, pp. 207–234, 2002.
- [37] S.-C. Lin, W. F. Punch III, and E. D. Goodman, "Coarse-grain parallel genetic algorithms: categorization and new approach," in *Sixth IEEE Symposium on Parallel and Distributed Processing*, pp. 28–37, IEEE, 1994.

APPENDIX

Here are some Description and Visualization of the dataset.

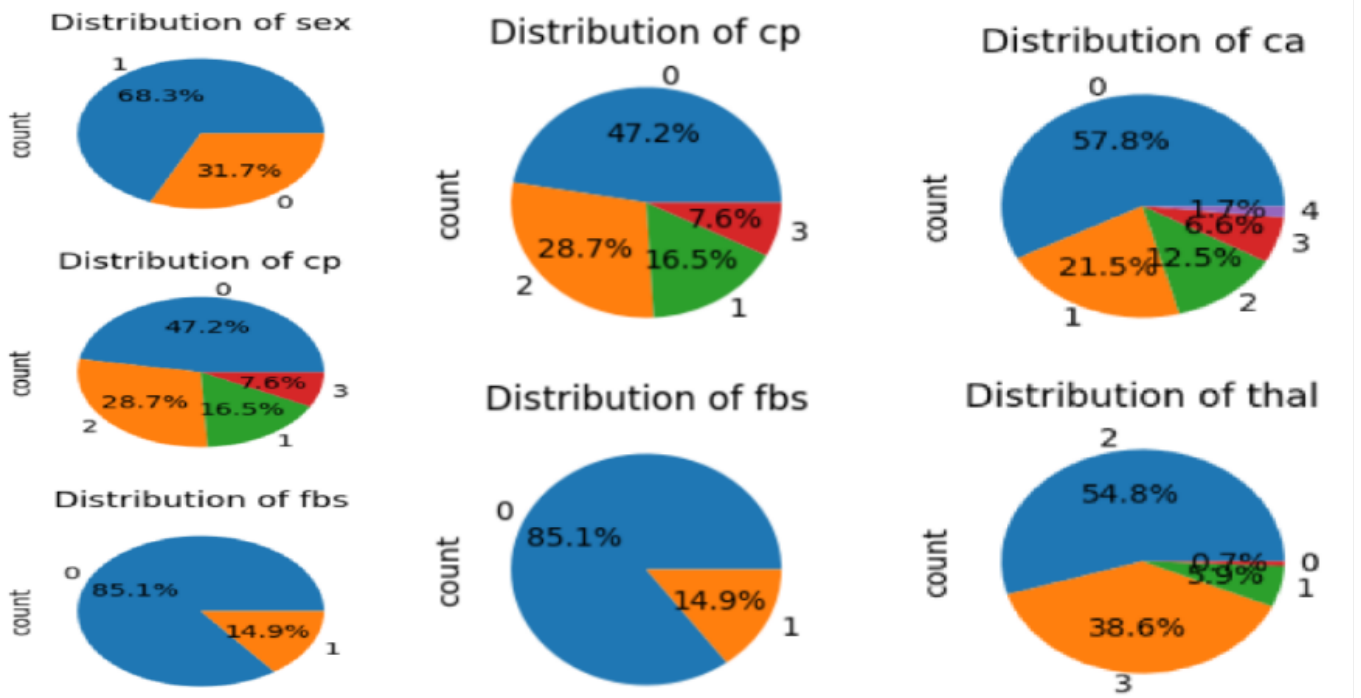


Fig. 2. Pie charts of Distribution of Categorical Variables

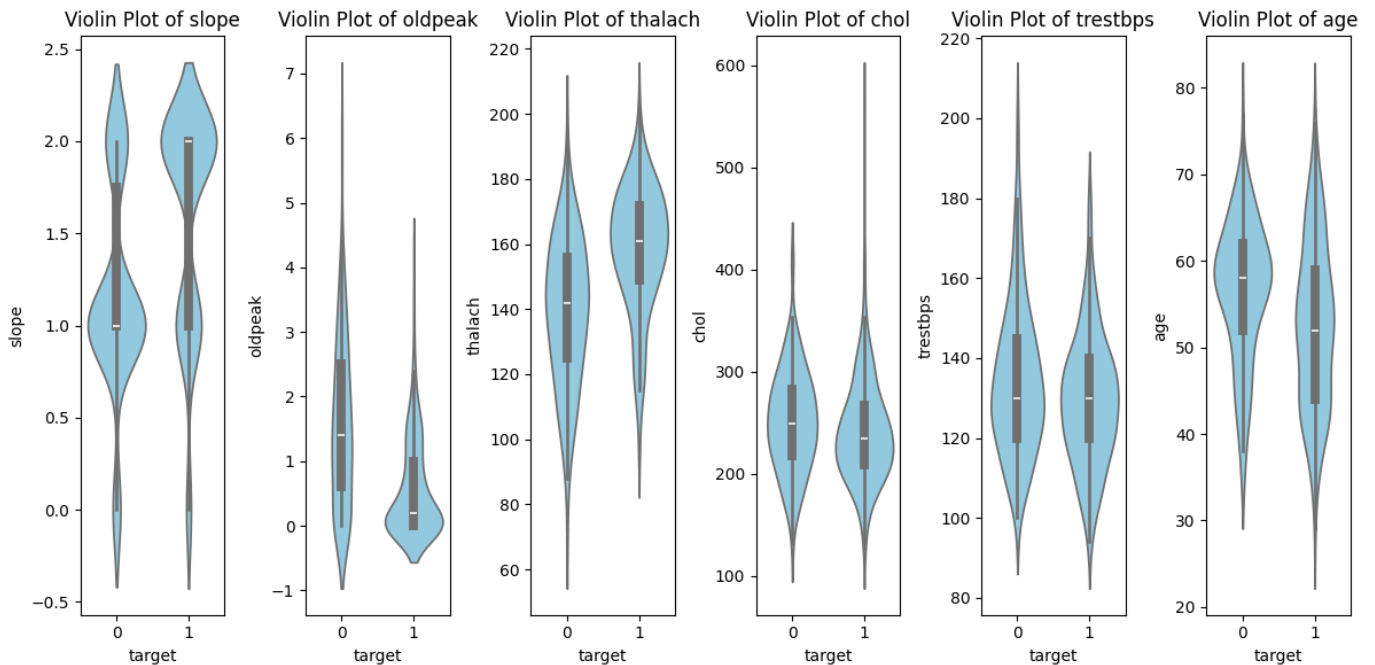


Fig. 3. Violin Plot of Continuous Variables